

CMPT 276 – Project Phase 3

Testing Report: Zombie Hamlet

Group - 8

Manya Asri - 301579204
Niki Saberi - 301608011
Amitoj Kochar - 301591083
Ajay Partap Singh Sekhon - 301600629

1. Introduction	1
2. Unit Testing	1
2.1 UI	1
2.2 Game Panel	1
2.3 Player	1
2.4 Key Handler	1
2.5 User Manual Panel	1
2.6 Food	1
2.7 Coin	1
2.8 Zombie	1
2.9 Zombie Overlap Logic	2
2.10 Game Panel Logic	2
2.11 EscapeGate	2
2.12 Trap	2
2.13 SoundManager	2
2.14 Position	2
2.15 StillEntity	2
2.16 TileManager	2
3. Integration Testing	2
3.1 UI	2
3.2 Player	3
3.3 Game Panel	3
3.4 User Manual Panel	3
3.5 Sound Integration	3
3.6 Key Input Integration	3
3.7 Game State Integration	3
3.8 Trap Damage Integration	3
3.9 Food Healing Integration	3
3.10 Coin Collection Integration	3
3.11 Player-Zombie Integration	4
4. Test Quality Measures	4
4.1 Quality Assurance Practices	4
4.2 Test Organization	4
5. Code Coverage Analysis	4
5.1 Coverage Results	4
5.2 Uncovered Code	4
6. Findings and Improvements	4
6.1 Bugs Found & Fixed	4
6.2 Code Improvements Made	5
7. Test Execution	5
7.1 Running Tests	5
8. Conclusion	5

1. Introduction

This document outlines the testing process and results for our **Zombie Hamlet** game. Throughout Phase 3, we implemented an extensive suite of unit and integration tests using JUnit 5 to validate the stability, correctness, and reliability of our code. Our primary objective was to achieve a strong test coverage with meaningful assertions that accurately reflect expected in-game behaviour.

2. Unit Testing

Unit tests were written to verify individual components in isolation. Below is a summary of the key areas tested.

2.1 UI

Test Class: `uiTest.java`

Features Tested : Checked that UI components load properly and respond to basic key presses without errors.

Total Test Cases : 17

2.2 Game Panel

Test Class: `GamePanelUnitTest.java`

Features Tested : Tested GamePanel setup, difficulty configuration, timers, drawing methods, tile checks, and basic player state handling.

Total Test Cases : 22

2.3 Player

Test Class: `PlayerTest`

Features Tested : Tested health changes, coin counting, position updates, boundary movement, collision flags, and basic image loading without running the full game.

Total Test Cases : 26

2.4 Key Handler

Test Class: `KeyHandlerTest.java`

Features Tested : Tested that each key press and release correctly updates its boolean flag and that no keys start pressed.

Total Test Cases : 10

2.5 User Manual Panel

Test Class: `UserManualPaneTest.java`

Features Tested : Tested panel construction, rendering, keyTyped/keyReleased handling, and background initialization.

Total Test Cases : 5

2.6 Food

Test Class: `FoodTest.java`

Features Tested : Verified that Food objects correctly heal the player, update their collected state, prevent double-healing, and safely handle null-player interactions.

Total Test Cases : 5

2.7 Coin

Test Class: `CoinTest.java`

Features Tested : Tested Coin initialization, valid/invalid value handling, collection logic, coin-count incrementing, prevention of duplicate collection effects, and null-player safety.

Total Test Cases : 5

2.8 Zombie

Test Class: `ZombieTest.java`

Features Tested : Covered Zombie construction, movement updates, speed and direction initialization, animation cycling, drawing in all directions, image loading, and overlap-avoidance behavior across multiple scenarios

Total Test Cases : 14

2.9 Zombie Overlap Logic

Test Class: `ZombieAvoidOverlapTest.java`

Features Tested : Specifically validated the avoidOverlap behavior when Zombies are close together, far apart, or interacting in a minimal list. Ensured proper positional adjustments and non-movement cases.

Total Test Cases : 2

2.10 Game Panel Logic

Test Class: `GamePanelLogicTest.java`

Features Tested : Verified map-boundary behavior and tile-collision correctness. Ensured that walkability checks properly handle out-of-bounds coordinates and collision flag conditions.

Total Test Cases : 2

2.11 EscapeGate

Test Class: `EscapeGateTest.java`

Features Tested: Verified correct initialization and modification of the gate's position via constructor and setter methods. Confirmed that the gate is non-blocking for movement and ensured the drawing routine executes without runtime exceptions.

Total Test Cases: 4

2.12 Trap

Test Class: `TrapTest.java`

Features Tested : Tested Trap arming/disarming logic, damage application to the player, cooldown-based rearming, invalid constructor input, null-player behavior, and prevention of repeated triggering before cooldown.

Total Test Cases : 7

2.13 SoundManager

Test Class: `SoundManagerTest.java`

Features Tested: Verified audio clip loading, volume control, and playback functions (play/loop/stop). Ensured distinct stopping mechanisms work correctly and confirmed graceful handling of null inputs.

Total Test Cases: 7

2.14 Position

Test Class: `PositionTest.java`

Features Tested: Verified coordinate encapsulation via getters and setters, and ensured correct string formatting. Tested equality logic and hash code consistency to ensure distinct and identical positions are compared

Total Test Cases: 7

2.15 StillEntity

Test Class: `StillEntityTest.java`

Features Tested: Verified initialization via objects/coordinates and error handling for null inputs. Tested collision detection logic with boundary checks, and confirmed default non-blocking behavior and string formatting.

Total Test Cases: 8

2.16 TileManager

Test Class: `TileManagerTest.java`

Features Tested: Verified map parsing, ensuring correct dimensions and numeric ID initialization. Confirmed asset loading and tested rendering stability under different game states (gate open vs. closed) to ensure the map draws without exceptions.

Total Test Cases: 9

3. Integration Testing

Integration tests focused on interactions between components.

3.1 UI

Test Class: `uiTest.java`

Features Tested : Tested menu navigation inside a frame and confirmed difficulty selections work as expected

Total Test Cases : 5

3.2 Player

Test Class: `PlayerTest.java`

Features Tested : Checked direction changes from key inputs, movement updates, drawing on a Graphics2D surface, and collision checks across all directions inside a GamePanel.

Total Test Cases : 6

3.3 Game Panel

Test Class: `GamePanelIntegrationTest.java`

Features Tested: Game thread execution, coin collection and gate opening, win condition triggering, player damage handling, state updates (play/win/game over), pause functionality, rendering in all states, game restart, menu navigation, collision detection (coin/trap/food), zombie behavior updates, food collection and removal.

Total Test Cases : 41

3.4 User Manual Panel

Test Class: `UserManualPaneTest.java`

Features Tested : Tested BACKSPACE navigation and ignored non-navigation keys.

Total Test Cases : 2

3.5 Sound Integration

Test Class: `SoundIntegrationTest.java`

Features Tested: SoundManager initialization, sound method execution, GamePanel integration with SoundManager

Total Test Cases: 3

3.6 Key Input Integration

Test Class: `KeyInputIntegrationTest.java`

Features Tested: Player movement in all directions, pause system, game state transitions with Enter key, menu navigation with Escape/Space, multiple key simultaneous presses, collision detection with movement

Total Test Cases: 14

3.7 Game State Integration

Test Class: `GameStateIntegrationTest.java`

Features Tested: Initial game state, state transitions between play/win/game over, coin-gate opening integration

Total Test Cases: 3

3.8 Trap Damage Integration

Test Class: `TrapDamageIntegrationTest.java`

Features Tested: Player damage from traps, trap disarming after triggering, damage value storage

Total Test Cases: 3

3.9 Food Healing Integration

Test Class: `FoodHealingIntegrationTest.java`

Features Tested: Food healing player health, non-blocking movement property, initial food state

Total Test Cases: 3

3.10 Coin Collection Integration

Test Class: `CoinCollectionIntegrationTest.java`

Features Tested: Coin collection increasing score, multiple coin collections, coin value storage, prevention of double collection, initial coin state

Total Test Cases: 5

3.11 Player-Zombie Integration

Test Class: `PlayerZombieIntegrationTest.java`

Features Tested: Player damage from zombie proximity, multiple zombie coexistence

Total Test Cases: 2

4. Test Quality Measures

4.1 Quality Assurance Practices

To make sure our tests are reliable and meaningful, we did:

- Used meaningful assertions that verify expected behavior, not just “no exceptions occur.”
- Gave tests clear, descriptive names so it’s easy to understand what each one does.
- Used `@AfterEach` to clean up UI and game panel elements after each test.
- Designed tests to be independent so one test doesn’t affect another (reduce coupling).
- Covered edge cases thoroughly, including boundaries, invalid inputs, and extreme values.
- Added comments to explain the purpose of each test for easier maintenance.

4.2 Test Organization

We structured our tests to be easy to read and maintain:

- Keep unit tests and integration tests separate using headings in our file.
- Grouped related tests together.
- Used helper methods (like `setupPanel()` and `cleanup()`) to reduce code repetition.
- Followed consistent formatting and structure across all test classes.

5. Code Coverage Analysis

5.1 Coverage Results

ZombieGame

Sessions

ZombieGame

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
ca.stu.cmp276.group8		94%		84%	89	422	124	1,405	6	147	1	22
Total	383 of 6,637	94%	83 of 530	84%	89	422	124	1,405	6	147	1	22

Created with JaCoCo 0.8.11.202310140853

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
GamePanel		96%		84%	27	116	26	439	0	25	0	1
ui.MenuPane		87%		67%	12	31	17	117	1	14	0	1
ui		51%		75%	3	6	21	45	2	3	0	1
Player		93%		82%	17	72	20	195	0	20	0	1
ui.StartPagePanel		90%		83%	1	9	9	72	0	6	0	1
Zombie		95%		80%	10	34	11	120	0	7	0	1
Coin		84%		85%	2	14	5	45	0	7	0	1
TileManager		94%		87%	2	12	5	46	0	4	0	1
Main		0%		n/a	3	3	5	5	3	3	1	1
Pathfinder		97%		71%	13	30	3	69	0	7	0	1
Trap		92%		85%	2	14	4	39	0	7	0	1
Food		94%		83%	2	14	4	37	0	8	0	1
SoundManager		96%		100%	0	13	4	46	0	6	0	1
UserManualPanel		100%		100%	0	6	0	40	0	5	0	1
KeyHandler		100%		100%	0	20	0	38	0	4	0	1
Position		100%		100%	0	13	0	16	0	8	0	1
StillEntity		100%		100%	0	9	0	18	0	7	0	1
MovingEntity		100%		n/a	0	1	0	2	0	1	0	1
Difficulty		100%		n/a	0	1	0	4	0	1	0	1
Node		100%		n/a	0	1	0	7	0	1	0	1
TileManager.Tile		100%		n/a	0	1	0	2	0	1	0	1
EscapeGate		100%		n/a	0	2	0	3	0	2	0	1
Total	414 of 6,637	93%	88 of 530	83%	94	422	134	1,405	6	147	1	22

5.2 Uncovered Code

1. The UI constructor cannot be tested because it initializes a `JFrame` which would create a visible window, blocking the test runner and requiring manual closure.
2. Any code paths that call `System.exit(0)` were skipped, as executing them would terminate the test runner.
3. Some image- and audio loading failure branches remain untested, since simulating resource failures is fragile without running the full game. These untested paths (catch statements) represent approximately 5% of total code and 0% of core game logic.
4. We did add a test for the UI, but to prevent the game from running in the background, certain interactive behaviors had to be skipped.

5. Key input handling for **ESC**, **ENTER**, and **SPACE** which trigger menu returns, game restarts, or program exit could not be safely tested in automated tests.

6. Findings and Improvements

6.1 Bugs Found & Fixed

- The `equals()` method in the **Position** class was updated to properly handle **null** objects. Although an `equals()` method already existed, it did not correctly check for **null**, which caused certain comparison tests to fail.
- Updating it allowed us to reliably test position matching and boundary cases. Some game objects (e.g., **Zombies**, **Coins**, **Traps**, and **Food** lists) were not accessible from test classes due to restrictive access modifiers. This prevented us from validating internal state changes in the **GamePanel**. We identified the issue when tests repeatedly failed to read collection contents.
- The first catch() bracket in `TileManager.java` had an `IOException`, which would not read other exception cases.

6.2 Code Improvements Made

1. Added a safer, more complete `equals()` implementation in **Position** and remove the older method to support null checks and ensure consistent comparison during unit tests:

```
@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (obj == null || getClass() != obj.getClass()) return false;
    Position that = (Position) obj;
    return this.row == that.row && this.col == that.col;
}
```

2. Adjusted access modifiers for several fields in **GamePanel**, such as:
 - `ArrayList<Zombie> zombies`
 - `ArrayList<Coin> coins`
 - `ArrayList<Trap> traps`
 - `ArrayList<Food> foodItems`
3. Fixed the catch statement condition in `TileManager.java` (changed from `IOException` to `Exception`).

7. Test Execution

7.1 Running Tests

Compile command: `mvn clean compile`

Run the tests: `mvn test`

To view Coverage result: `open target/site/jacoco/index.html`

To view Javadoc comments: `mvn javadoc:javadoc && open target/reports/apidocs/index.html`

8. Conclusion

In Phase 3, we built a strong and reliable testing framework for the Zombie Hamlet game. Our combined unit and integration tests covered all major gameplay systems, including movement, collisions, UI interactions, sound, and game-state transitions. These tests helped us verify expected behaviour, identify hidden bugs, and improve the quality of several core classes. The improvements we made such as fixing comparison logic, correcting exception handling, and adjusting access modifiers directly increased the stability and consistency of the game. With high coverage and well organized tests, our project is now easier to maintain and safer to extend in future phases.

Overall, this testing phase strengthened both the game and our development practices, ensuring that Zombie Hamlet performs reliably across a wide range of scenarios.